

VWA: Hardware Efficient Vectorwise Accelerator for Convolutional Neural Network

Kuo-Wei Chang^{ID} and Tian-Sheuan Chang^{ID}, *Senior Member, IEEE*

Abstract—Hardware accelerators for convolution neural networks (CNNs) enable real-time applications of artificial intelligence technology. However, most of the existing designs suffer from low hardware utilization or high area cost due to complex data flow. This paper proposes a hardware efficient vectorwise CNN accelerator that adopts a 3×3 filter optimized systolic array using 1-D broadcast data flow to generate partial sum. This enables easy reconfiguration for different kinds of kernels with interleaved input or elementwise input data flow. This simple and regular data flow results in low area cost while attains high hardware utilization. The presented design achieves 99%, 97%, 93.7%, and 94% hardware utilization for VGG-16, ResNet-34, GoogLeNet, and Mobilenet, respectively. Hardware implementation with TSMC 40nm technology takes 266.9K NAND gate count and 191KB SRAM to support 168GOPS throughput while consumes only 154.98mW when running at 500MHz operating frequency, which has superior area and power efficiency than other designs.

Index Terms—Convolution neural networks (CNNs), hardware design, accelerators.

I. INTRODUCTION

DEEP convolution neural networks (DCNNs) have been widely used in computer vision tasks, such as recognition [1]–[5], detection [6]–[10], and autonomous vehicles during recent years for its significant improvement over traditional approaches. However, the computation of CNNs demands a lot of multiplications and accumulations (MACs) and millions of data amount per layer that prohibit its wide usage in real-time applications. Thus, hardware acceleration for DCNNs is demanded to satisfy the computation and bandwidth requirements under the real time constraint.

Various hardware accelerators have been proposed recently [11]–[23], [33]–[38]. References [11]–[15] minimize the communication traffic by a tiling strategy and internal buffers. However, the modern DCNNs are too large to store all weights into the on-chip memory. Reference [16] adopts a spatial array architecture and row stationary data flow that helps maximize data reuse but leads to high processing

element (PE) cost due to large local storage. In addition, its fixed PE configuration results in low hardware utilization. Its improved version [17] has higher hardware utilization and supports sparse CNNs by clusters to increase throughput, but needs large area overhead. Reference [18] proposes a systolic array architecture with full reconfigurations for different convolutional kernels. It achieves high hardware utilization but needs significant PE area cost due to complex control overhead. The design in [19] is precision scalable to achieve low power consumption under different application scenarios. Reference [20] uses a filter-type structure that can be reconfigured to fit different needs but this filter-like data flow limits the flexibility of reconfigurations. The streaming architecture in [21] reduces data movement through filter reuse and input reuse to optimize energy efficiency, but it has low hardware utilization beyond optimized 3×3 architectures. The systolic array architecture in [22] is also a reconfigurable design to fit different convolution kernels but with a propagated input data flow that results in long latency and low hardware utilization. Reference [23] proposes an accelerator for image segmentation which supports dilated and transposed convolution. In addition, it cuts down the redundant zero computations for higher throughput. Reference [31] provides near non-blocking communication via reconfigurable links with high bandwidth to support efficient irregular data flow mapping and provide high utilization of computing units. Reference [32] proposes a low precision design that uses heterogeneous representation and significant bits securing encoding which can exploit the advantage of both efficiency of fixed-point and flexibility of floating-point to optimize their PE architecture and consume less power and area. Reference [33] adopts different combinations of input, weight and output data flow to optimize their design. The data flow in [34] inspired from the fully-connected layer can accelerate convolutional processes and parametrize design from low memory accesses to low latency systems. Reference [35] is a computing in memory design that adopts SRAM array for vector matrix multiplication for low power deep learning applications. Reference [36] implements convolutions of different kernel sizes with multiple parallel fast FIR based convolution units. Although existing designs can support most of kernel sizes and stride lengths, they suffer from low hardware utilization or high hardware cost due to constrained data flow or complex PE structures for such reconfiguration requirements.

To achieve high hardware utilization with low area overhead, this paper proposes a hardware efficient vectorwise

Manuscript received June 14, 2019; revised August 27, 2019; accepted September 15, 2019. Date of publication October 2, 2019; date of current version January 15, 2020. This work was supported in part by the Ministry of Science and Technology, Taiwan, under Grant 108-2634-F-009-005 and Grant 107-2119-M-009-019 and in part by the Research of Excellence Program under Grant 106-2633-E-009-001. This article was recommended by Associate Editor P. K. Meher. (Corresponding author: Kuo-Wei Chang.)

The authors are with the Department Electronics Engineering, National Chiao Tung University, Hsinchu 30010, Taiwan (e-mail: andy0703.ee05g@nctu.edu.tw; tschang@mail.nctu.edu.tw).

Color versions of one or more of the figures in this article are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCSI.2019.2942529

1549-8328 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

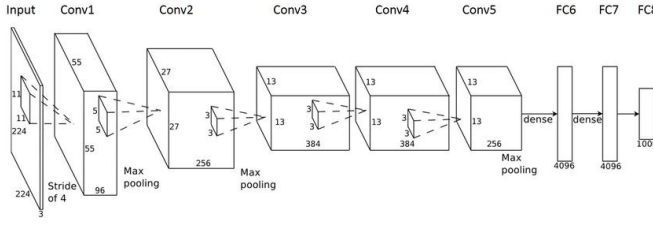


Fig. 1. A typical CNN model, Alexnet [1].

accelerator based on a PE array for CNN. This design adopts broadcasted vectorwise input and weight for the PE array to avoid local weight storage. The array is partitioned into blocks and optimized for the widely used 3×3 convolutions but also easily reconfigured for other kernel sizes due to the broadcasted data flow. The PE array has high hardware utilization even for the non-unit stride case by the interleaved input. This design can also support 1×1 convolution commonly used in the low complexity network with the elementwise input data flow. Even with above versatile reconfigurations, the proposed design still has high hardware efficiency compared to other designs.

The rest of the paper is organized as following. Section II gives a short overview of the CNN algorithm. Section III presents the proposed architecture. Section IV shows the details of the data flow for all kinds of convolutional filters supported in our architecture. The implementation results and comparisons are shown in Section V. Finally, we conclude this paper in Section VI.

II. OVERVIEW OF CNN

Fig. 1 shows a typical DCNN model, called AlexNet [1], that classifies a 224×224 RGB image into one of 1000 categories. A typical CNN structure consists of the convolutional layers, pooling layers, fully-connected layers, and the activation function. The convolutional layers transform images into highly abstract representations called feature maps. The pooling layers execute down-sampling operations to reduce feature map size and improve the translational invariance of features. In the state-of-the-art networks, the pooling layers could be replaced by layers of 3×3 convolution with stride two. The commonly used activation function like rectified linear unit (ReLU) adds non-linearity after the convolutional layers or the fully-connected layers to improve feature representations. The fully-connected layers classify the image based on those extracted features, which is a pure neural network with a lot of weight numbers.

The convolutional layer applies a filter kernel to the whole image, which can be defined as

$$O(b, c, r, w) = Act\left(\sum_{u=0}^{U-1} \sum_{i=0}^{I-1} \sum_{j=0}^{J-1} F(b, u, Sr + i, Sw + j) \times W(c, u, i, j) + B(u)\right) \quad (1)$$

where O , F , W , and B are the matrices of output feature maps, input feature maps, weight, and bias, respectively. In which, Act is the activation function, such as ReLU, b

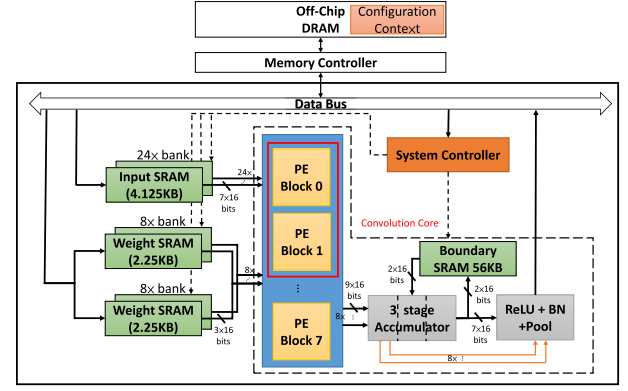


Fig. 2. The proposed system architecture.

is the image batch size, c is the number of filters, r is the output row size, w is the output column size, u is the input channel, i is the weight row size, j is the weight column size, and S is the stride length. For this whole image operation, the computation needs $c \times u \times i \times j \times r \times w \times b$ MACs for one layer. The computation cost of the whole model is increased significantly with modern deeper and wider models, which will be over tens or hundreds of giga or tera operations per second for models like ResNet and GoogLeNet [37].

III. SYSTEM ARCHITECTURE

A. Overview

Fig. 2 shows the proposed system architecture, which consists of eight PE blocks, three-stage accumulators and a boundary data SRAM buffer for convolution computation, a post-processing module for ReLU, batch-normalization and pooling, and multi-bank input and weight SRAM buffers for continuous data access from external memory. The weight data buffer is a ping-pong buffer, which is divided into eight banks for eight PEs, respectively. The input buffer is divided into 24 banks for eight PEs. Each PE block will connect to three input SRAM banks to support different filter sizes.

The proposed design will first load a tile of input map and weight data to the local SRAM buffers for data reuse. Each PE block will use one channel of input map and one weight filter to compute partial convolution results. With eight PEs, we will be able to process eight input channels and eight filters at the same time. The partial convolution results from all PE blocks will be stored and accumulated locally through the three stage accumulators to avoid external memory access. During the above accumulation, partial results at the top and bottom boundary of tiles will be also stored in the boundary buffer to avoid repeated computation at the tile boundary. The final convolutional result will be processed by the post-processing module for ReLU, batch-normalization, and pooling operations, and then stored to external memory.

B. PE Blocks

Fig. 3 illustrates the 7×3 MAC array design of one PE block. The PE block has 3×7 inputs broadcasted horizontally and three weights broadcasted vertically that are multiplied

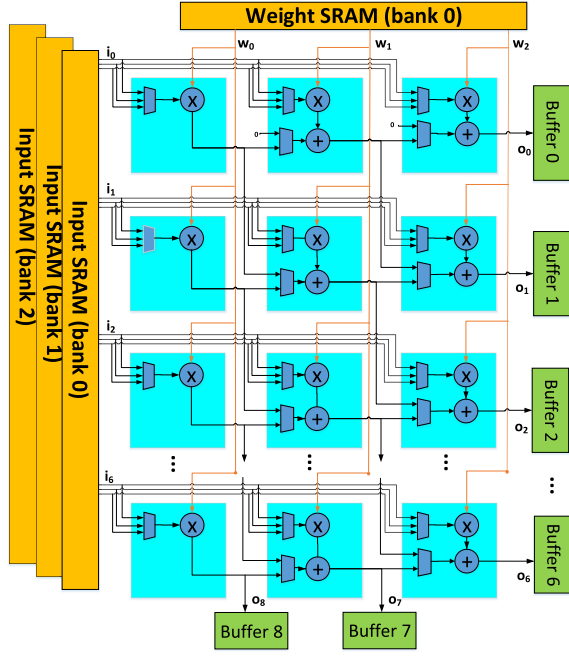


Fig. 3. The architecture of the PE block. In which, 3×7 input data and 3 weight data are broadcasted to a 7×3 MAC array and generate 9 output data.

together and then summed along horizontal direction for an 1×1 filter or diagonal direction for other filter sizes. To support different convolution filters, each input row has three inputs so that each MAC can select the desired input via a 3-to-1 multiplexer. This design chooses a three-column width for weights to optimize computations of 3×3 convolutions and avoid long critical path. With this proposed data flow, we can maximize hardware utilization while keep data flow regular for lower area cost.

Beyond the above block level reconfiguration, the proposed eight PE blocks can also be reconfigured as two configurations, (8, 7, 3): 8 blocks, and 7 rows by 3 columns per block, or (4, 14, 3): 4 blocks, and 14 rows by 3 columns per block that combines two PE blocks into one as in Fig. 2. This array level reconfiguration can support different input map sizes while maximize hardware utilization. For example, the PE array is reconfigured as (4, 14, 3) in the first layer of the models for image recognition, such as AlexNet, VGGNet, and ResNet because the input map only consists of three channels with R, G, and B. But the PE array can be reconfigured as (8, 7, 3) in other layers due to wider channel numbers.

C. Accumulator

Fig. 4 shows the accumulator that consists of a three-stage pipeline structure to accumulate PE output along the channel dimension. For clarity of explanation, we will assume a 3×3 filter in the rest of this subsection. As shown in Fig. 3, each PE block has nine outputs, which are partial sums of one filter column for one channel. For a 3×3 filter, three partial sums of the same channel will be accumulated at the first stage. Then, the filtered output from eight channels (that is, from eight PE blocks) will be summed together with a tree adder

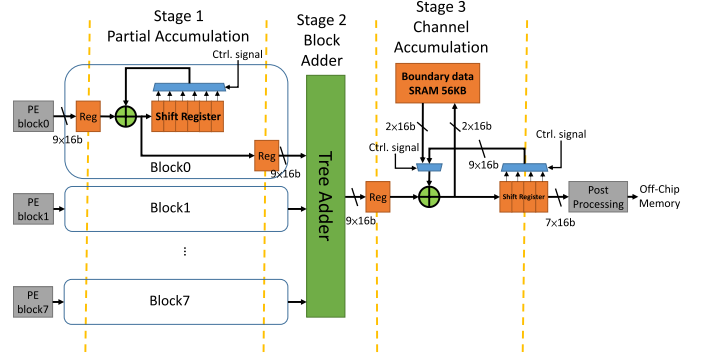


Fig. 4. The pipelined accumulator.

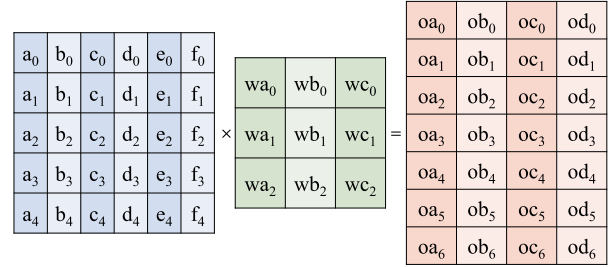


Fig. 5. A convolution example with 5×6 input, 3×3 weight, and 7×4 output.

at the second stage. For channel number larger than eight, they are divided into groups of eight channel processing in the PE blocks. Then, these different groups of channels will be accumulated at the third stage to generate the convolution output. If these output data are at the tile boundary, they will be stored in the global boundary data SRAM and then accumulated later with the output from the neighboring tile output. This accumulator is reconfigurable to fit the needs of different convolution kernels as illustrated in the later section.

IV. DATA FLOW OF DIFFERENT KERNELS

The proposed accelerator is designed to optimize computations of 3×3 filters stride one with a vectorwise scheduling due to its wide usage. For larger filter size, we adopt kernel decomposition method [20] so that this design only needs extra support of 4×4 , and 5×5 filters. In addition, with the popularity of low complexity models, this design also supports 3×3 with stride two by interleaved input, depthwise convolution by reconfigurable accumulators and 1×1 filter by elementwise input data flow for higher hardware utilization.

A. Basic 3×3 Convolution With Unit Stride

Fig. 6 shows the data flow of a PE block to compute an example with 5×6 input tile and a 3×3 filter kernel as in Fig. 5. This example case assumes a 5×3 MAC array for clarity. In which, the blocks with the same color are the partial results for the same convolution output. Above computation also includes tile boundary cases on top (e.g. $oa_0 - oa_1$, $ob_0 - ob_1$, $oc_0 - oc_1$, $od_0 - od_1$) and bottom (e.g. $oa_5 - oa_6$,

t=1	wa ₀	wa ₁	wa ₂	Output	t=4	wa ₀	wa ₁	wa ₂	Output	t=7	wa ₀	wa ₁	wa ₂	Output	t=10	wb ₀	wb ₁	wb ₂	Output
a ₀	a ₀ wa ₀	a ₀ wa ₁	a ₀ wa ₂	oa ₀	c ₀	c ₀ wa ₀	c ₀ wa ₁	c ₀ wa ₂	oc ₀	d ₀	d ₀ wa ₀	d ₀ wa ₁	d ₀ wa ₂	od ₀	e ₀	e ₀ wb ₀	e ₀ wb ₁	e ₀ wb ₂	od ₀
a ₁	a ₁ wa ₀	a ₁ wa ₁	a ₁ wa ₂	oa ₁	c ₁	c ₁ wa ₀	c ₁ wa ₁	c ₁ wa ₂	oc ₁	d ₁	d ₁ wa ₀	d ₁ wa ₁	d ₁ wa ₂	od ₁	e ₁	e ₁ wb ₀	e ₁ wb ₁	e ₁ wb ₂	od ₁
a ₂	a ₂ wa ₀	a ₂ wa ₁	a ₂ wa ₂	oa ₂	c ₂	c ₂ wa ₀	c ₂ wa ₁	c ₂ wa ₂	oc ₂	d ₂	d ₂ wa ₀	d ₂ wa ₁	d ₂ wa ₂	od ₂	e ₂	e ₂ wb ₀	e ₂ wb ₁	e ₂ wb ₂	od ₂
a ₃	a ₃ wa ₀	a ₃ wa ₁	a ₃ wa ₂	oa ₃	c ₃	c ₃ wa ₀	c ₃ wa ₁	c ₃ wa ₂	oc ₃	d ₃	d ₃ wa ₀	d ₃ wa ₁	d ₃ wa ₂	od ₃	e ₃	e ₃ wb ₀	e ₃ wb ₁	e ₃ wb ₂	od ₃
a ₄	a ₄ wa ₀	a ₄ wa ₁	a ₄ wa ₂	oa ₄	c ₄	c ₄ wa ₀	c ₄ wa ₁	c ₄ wa ₂	oc ₄	d ₄	d ₄ wa ₀	d ₄ wa ₁	d ₄ wa ₂	od ₄	e ₄	e ₄ wb ₀	e ₄ wb ₁	e ₄ wb ₂	od ₄
Output	oa ₀	oa ₁	oa ₂		Output	oc ₀	oc ₁	oc ₂		Output	od ₀	od ₁	od ₂		Output	od ₀	od ₁	od ₂	

t=2	wa ₀	wa ₁	wa ₂	Output	t=5	wb ₀	wb ₁	wb ₂	Output	t=8	wb ₀	wb ₁	wb ₂	Output	t=11	wc ₀	wc ₁	wc ₂	Output
b ₀	b ₀ wa ₀	b ₀ wa ₁	b ₀ wa ₂	ob ₀	c ₀	c ₀ wb ₀	c ₀ wb ₁	c ₀ wb ₂	ob ₀	d ₀	d ₀ wb ₀	d ₀ wb ₁	d ₀ wb ₂	oc ₀	e ₀	e ₀ wc ₀	e ₀ wc ₁	e ₀ wc ₂	oc ₀
b ₁	b ₁ wa ₀	b ₁ wa ₁	b ₁ wa ₂	ob ₁	c ₁	c ₁ wb ₀	c ₁ wb ₁	c ₁ wb ₂	ob ₁	d ₁	d ₁ wb ₀	d ₁ wb ₁	d ₁ wb ₂	oc ₁	e ₁	e ₁ wc ₀	e ₁ wc ₁	e ₁ wc ₂	oc ₁
b ₂	b ₂ wa ₀	b ₂ wa ₁	b ₂ wa ₂	ob ₂	c ₂	c ₂ wb ₀	c ₂ wb ₁	c ₂ wb ₂	ob ₂	d ₂	d ₂ wb ₀	d ₂ wb ₁	d ₂ wb ₂	oc ₂	e ₂	e ₂ wc ₀	e ₂ wc ₁	e ₂ wc ₂	oc ₂
b ₃	b ₃ wa ₀	b ₃ wa ₁	b ₃ wa ₂	ob ₃	c ₃	c ₃ wb ₀	c ₃ wb ₁	c ₃ wb ₂	ob ₃	d ₃	d ₃ wb ₀	d ₃ wb ₁	d ₃ wb ₂	oc ₃	e ₃	e ₃ wc ₀	e ₃ wc ₁	e ₃ wc ₂	oc ₃
b ₄	b ₄ wa ₀	b ₄ wa ₁	b ₄ wa ₂	ob ₄	c ₄	c ₄ wb ₀	c ₄ wb ₁	c ₄ wb ₂	ob ₄	d ₄	d ₄ wb ₀	d ₄ wb ₁	d ₄ wb ₂	oc ₄	e ₄	e ₄ wc ₀	e ₄ wc ₁	e ₄ wc ₂	oc ₄
Output	ob ₀	ob ₁	ob ₂		Output	ob ₀	ob ₁	ob ₂		Output	oc ₀	oc ₁	oc ₂		Output	oc ₀	oc ₁	oc ₂	

t=3	wb ₀	wb ₁	wb ₂	Output	t=6	wc ₀	wc ₁	wc ₂	Output	t=9	wc ₀	wc ₁	wc ₂	Output	t=12	wc ₀	wc ₁	wc ₂	Output
b ₀	b ₀ wb ₀	b ₀ wb ₁	b ₀ wb ₂	oa ₀	c ₀	c ₀ wc ₀	c ₀ wc ₁	c ₀ wc ₂	oa ₀	d ₀	d ₀ wc ₀	d ₀ wc ₁	d ₀ wc ₂	ob ₀	f ₀	f ₀ wc ₀	f ₀ wc ₁	f ₀ wc ₂	od ₀
b ₁	b ₁ wb ₀	b ₁ wb ₁	b ₁ wb ₂	oa ₁	c ₁	c ₁ wc ₀	c ₁ wc ₁	c ₁ wc ₂	oa ₁	d ₁	d ₁ wc ₀	d ₁ wc ₁	d ₁ wc ₂	ob ₁	f ₁	f ₁ wc ₀	f ₁ wc ₁	f ₁ wc ₂	od ₁
b ₂	b ₂ wb ₀	b ₂ wb ₁	b ₂ wb ₂	oa ₂	c ₂	c ₂ wc ₀	c ₂ wc ₁	c ₂ wc ₂	oa ₂	d ₂	d ₂ wc ₀	d ₂ wc ₁	d ₂ wc ₂	ob ₂	f ₂	f ₂ wc ₀	f ₂ wc ₁	f ₂ wc ₂	od ₂
b ₃	b ₃ wb ₀	b ₃ wb ₁	b ₃ wb ₂	oa ₃	c ₃	c ₃ wc ₀	c ₃ wc ₁	c ₃ wc ₂	oa ₃	d ₃	d ₃ wc ₀	d ₃ wc ₁	d ₃ wc ₂	ob ₃	f ₃	f ₃ wc ₀	f ₃ wc ₁	f ₃ wc ₂	od ₃
b ₄	b ₄ wb ₀	b ₄ wb ₁	b ₄ wb ₂	oa ₄	c ₄	c ₄ wc ₀	c ₄ wc ₁	c ₄ wc ₂	oa ₄	d ₄	d ₄ wc ₀	d ₄ wc ₁	d ₄ wc ₂	ob ₄	f ₄	f ₄ wc ₀	f ₄ wc ₁	f ₄ wc ₂	od ₄
Output	oa ₀	oa ₁	oa ₂		Output	oa ₀	oa ₁	oa ₂		Output	ob ₀	ob ₁	ob ₂		Output	od ₀	od ₁	od ₂	

Fig. 6. Dataflow chart for the example in Fig. 5 with 5×3 MACs. In which, the same color elements belong to the same filter output, which will be summed together in a PE at the same cycle and then accumulated later in the accumulator at different cycles, e.g. red elements at $t = 2, 5$, and 9 accumulated for ob_2 .

$$\begin{aligned}
 oa &= a \times wa + b \times wb + c \times wc \\
 ob &= b \times wa + c \times wb + d \times wc \\
 oc &= c \times wa + d \times wb + e \times wc \\
 od &= d \times wa + e \times wb + f \times wc
 \end{aligned}$$

Fig. 7. The computational order for 3×3 convolutions with unit stride, where oa to od are the column vector of the output. a to f are the column vector of the input. wa to wc are the column vector of the weight. Numbers 1-12 represent cycle numbers in Fig. 6.

$$\begin{aligned}
 \text{Block 0} \quad oa_0 &= a \times wa_{0-2} + b \times wb_{0-2} + c \times wc_{0-2} + d \times wd_{0-2} \\
 ob_0 &= b \times wa_{0-2} + c \times wb_{0-2} + d \times wc_{0-2} + e \times wd_{0-2} \\
 oc_0 &= c \times wa_{0-2} + d \times wb_{0-2} + e \times wc_{0-2} + f \times wd_{0-2} \\
 \text{Block 1} \quad oa_1 &= a \times wa_{3-4} + b \times wb_{3-4} + c \times wc_{3-4} + d \times wd_{3-4} \\
 ob_1 &= b \times wa_{3-4} + c \times wb_{3-4} + d \times wc_{3-4} + e \times wd_{3-4} \\
 oc_1 &= c \times wa_{3-4} + d \times wb_{3-4} + e \times wc_{3-4} + f \times wd_{3-4}
 \end{aligned}$$

Fig. 8. The computational order for 4×4 convolutions with unit stride, where wa_{0-2} is a vector with wa_0, wa_1, wa_2 , wb_{0-2} is a vector with wb_0, wb_1, wb_2 , and so on. $oa = oa_0 + oa_1$, $ob = ob_0 + ob_1$, $oc = oc_0 + oc_1$.

$ob_5 - ob_6, oc_5 - oc_6, od_5 - od_6$), which will be stored in the global boundary buffer. For a 3×3 convolution, one column vector of input is broadcasted horizontally along the MAC row (e.g. $a_0 - a_4$ when $t = 1$) and one column vector of filter weight is broadcasted vertically (e.g. $wa_0 - wa_2$ when $t = 1$). With this broadcasted data flow, the multiplication result along the diagonal direction will belong to the same output pixels (e.g. $oa_0 - oa_6$ when $t = 1$), which will be summed together in the same cycle as the multiplications occur, and then accumulated sequentially in the following accumulator. This vectorwise scheduling enables simple data flow for high hardware utilization and low area cost. For 3×3 depthwise convolution, the output of the first stage of the accumulators will be the desired result.

The overall vectorwise scheduling is shown in Fig. 6, which takes 12 cycles to complete the example case. The concept of

$$\begin{aligned}
 \text{Block 0} \quad oa_0 &= a \times wa_{0-2} + b \times wb_{0-2} + c \times wc_{0-2} + d \times wd_{0-2} + e \times we_{0-2} \\
 ob_0 &= b \times wa_{0-2} + c \times wb_{0-2} + d \times wc_{0-2} + e \times wd_{0-2} + f \times we_{0-2} \\
 \text{Block 1} \quad oa_1 &= a \times wa_{3-4} + b \times wb_{3-4} + c \times wc_{3-4} + d \times wd_{3-4} + e \times we_{3-4} \\
 ob_1 &= b \times wa_{3-4} + c \times wb_{3-4} + d \times wc_{3-4} + e \times wd_{3-4} + f \times we_{3-4}
 \end{aligned}$$

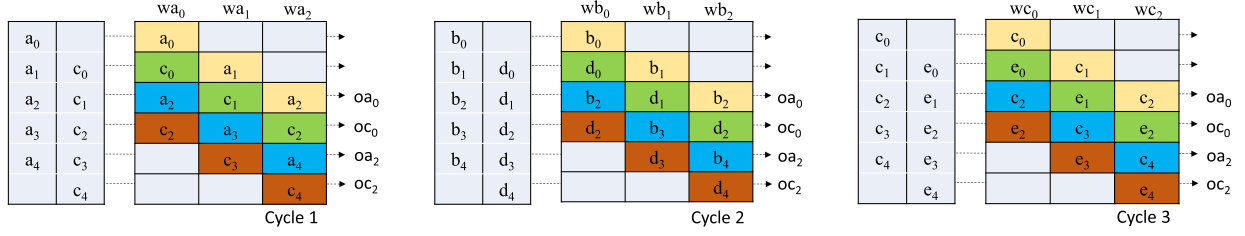
Fig. 9. The computational order for 5×5 convolutions with unit stride, where wa_{3-4} is a vector with wa_3, wa_4 , wb_{3-4} is a vector with wb_3, wb_4 , and so on. $oa = oa_0 + oa_1$, $ob = ob_0 + ob_1$.

$$\begin{bmatrix} a_0 & b_0 & c_0 & d_0 & e_0 \\ a_1 & b_1 & c_1 & d_1 & e_1 \\ a_2 & b_2 & c_2 & d_2 & e_2 \\ a_3 & b_3 & c_3 & d_3 & e_3 \\ a_4 & b_4 & c_4 & d_4 & e_4 \end{bmatrix} \times \begin{bmatrix} wa_0 & wb_0 & wc_0 \\ wa_1 & wb_1 & wc_1 \\ wa_2 & wb_2 & wc_2 \end{bmatrix} = \begin{bmatrix} oa_0 & oc_0 \\ oa_2 & oc_2 \end{bmatrix}$$

Fig. 10. An example of 3×3 convolutions/stride 2 with 5×5 input to generate 2×2 output.

this scheduling is to decompose a convolution into sums of MAC results by an input column and a weight column as shown in Fig. 7. Each MAC result of an input column and a weight column will take one cycle to complete. Thus, a 3×3 filter computation will need three cycles. However, these three cycles are not continuous. We rearrange the computation order as shown in Fig. 7 so that the same input can be reused successively for the computation to save the power of input buffer SRAM. Besides, this reorder enables a simple continuous addressing of the input SRAM buffer to reduce control overhead.

With the above computation, the speedup for the above example is $(\text{number of MACs}) / \text{cycles} = 180 / 12 = 15$,

Fig. 11. Flow chart for 3×3 convolutions with stride 2 example in Fig. 10.

which is equal to the number of PEs ($3 \times 5 = 15$). Thus, the hardware is fully utilized.

B. 4×4 and 5×5 Convolution With Unit Stride

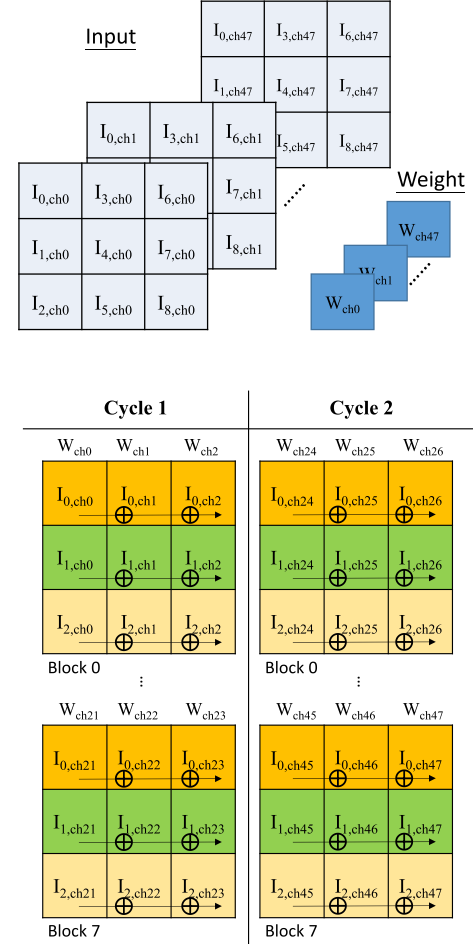
Beyond 3×3 convolution with unit stride, our proposed architecture also supports 4×4 and 5×5 convolutions to implement different filter sizes in current popular models. The 4×4 and 5×5 convolutions will take two PE blocks. As shown in Fig. 8 and Fig. 9, the first three elements (e.g. $wa_0 - wa_2$, $wd_0 - wd_2$ in 4×4 , or $wa_0 - wa_2$, $we_0 - we_2$ in 5×5) of the filter columns will be at the first block and the remaining elements (e.g. wa_3 , wd_3 in 4×4 , or $wa_3 - wa_4$, $we_3 - we_4$ in 5×5) will be at the second block with two or one column of unused MACs. The overall data flow is the same as the one in the 3×3 convolution but with properly weight assignments for 4×4 and 5×5 . With this approach, the hardware utilization will be lower since part of the PE block will not be used. This design tradeoff can make hardware regular since these kernel sizes are only used in the first input layer in most of the current network models.

C. 3×3 Convolution With Stride 2

For a 3×3 convolution with stride 2, a naive data flow implies a 25% of hardware utilization since only 25% of PE result is useful for the final result. To increase hardware utilization, we propose interleaved input as shown in Fig. 11. Thus, we add a multiplexer at the input data of multiplier in each PE as shown in Fig. 3 to support this. Fig. 11 shows the data flow of a PE block for an example with a 5×5 input, and 3×3 filter kernel with stride 2. Input data from two different input columns (e.g. $a_0 - a_4$, $c_0 - c_4$) will be interleaved as the input of successive PE rows to increase hardware utilization. An example as shown in Fig. 10 with stride 2 only need 3 cycles to generate oa_0 , oa_2 , oc_0 , and oc_2 . The result will be accumulated at the accumulator as 3×3 convolution with unit stride but with interleaved results. Thus, the speedup is $(\text{number of MACs})/(\text{cycles}) = 36/3 = 12$. However, the PE block will have some MACs unused due to interleaved input. So the utilization is $\text{speedup}/(\text{number of PEs}) = 12/18 = 67\%$ in this example, which is lower than that in 3×3 convolutions with unit stride but higher than that with a naive mapping.

D. 1×1 Convolution

1×1 convolutions are getting popular in modern deep learning models. However, unlike other convolutions, the 1×1

Fig. 12. Flow chart for 1×1 convolutions. The same color elements will be summed together.

convolution only has size one in the spatial dimension, which will result in much more output than 3×3 convolutions if we implement it naively on the current 3×3 convolution based PE blocks. This implies more parallel hardware needed for the following accumulators to keep PE array full utilized or the PE array stalled during the accumulation.

To solve above problems, we propose the elementwise input scheduling for 1×1 convolutions as shown in Fig. 12 that will have distinctive input elements in each multiplier. In this scheduling, each PE block will select their specific input feature map via a 3-to-1 multiplexer. With such input, the summation is now along the horizontal direction. Thus, we will have output number equal to $\text{block number} = 8$

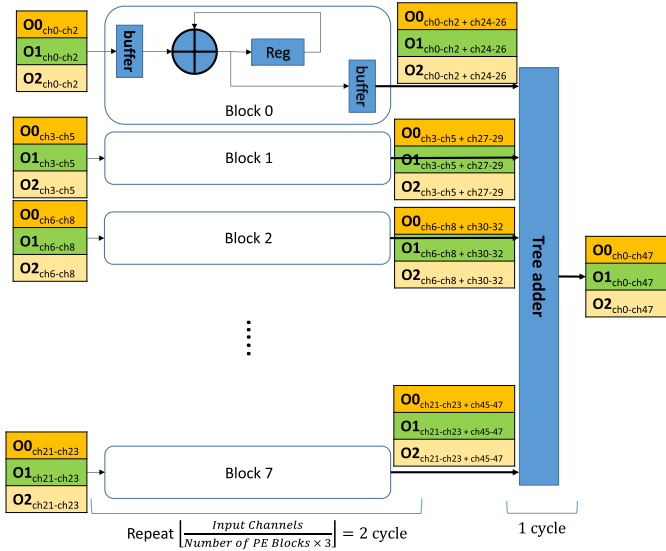


Fig. 13. Accumulation diagram for 1×1 convolutions. $O0_{ch0-ch2}$ represents that one output pixel have been accumulated with input channels from 0 to 2 in the stage.

for eight PE blocks. Each output from a PE block will have sum of three channel multiplications. In this flow, all PE outputs are just different parts of a channel output. Thus, we will accumulate them with the first two stages of the accumulator as shown in Fig. 13. The stage three is skipped in this case. With above flow, we can achieve high hardware utilization.

V. EXPERIMENTAL RESULT

A. Experiment Setup

This design has been implemented with the TSMC 40nm CMOS technology. The hardware power consumption is estimated by Synopsys PrimeTime PX. We also build a cycle based tool by Pytorch to properly evaluate hardware utilization, speedup and DRAM access under different models and PE array configurations. In addition, this tool also helps generate configuration information to configure the hardware controller. For the following analysis, we will use four famous models, VGG-16 [26], MobileNet [27], ResNet-34 [4], and GoogLeNet [28], trained on the ImageNet dataset as our test cases.

B. Implementation Result

Fig. 14 shows the chip layout and its implementation result. The core area is $1.25\text{mm} \times 1.25\text{mm}$ with 191KB SRAM including 99KB input SRAM bank, 36KB weight SRAM bank, and 56KB global boundary data SRAM. The peak performance is 168 GOPS at 500MHz based on the technology library from the worst PVT corner, when the PE utilization is 100%. The power consumption is 154.98mW, measured by running VGG-16 convolutions based on the technology library from the best PVT corner at 0.99 V power supply.

C. Memory Access and Energy Consumption Analysis

The external memory access of this design is reduced by data reuse. The input is divided into configurable tile size

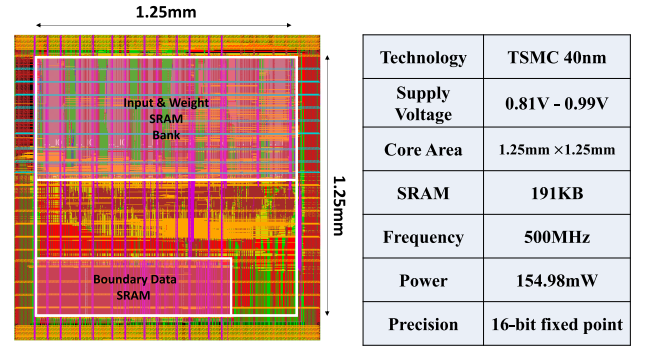


Fig. 14. Implementation result.

TABLE I
PER LAYER DRAM ACCESS AMOUNT FOR VGG-16

Layers	Input Access (MB)	Weight Access (MB)	Output Access (MB)	Total Access (MB)	Total Energy (mJ)	Tile Size (7×n)
1	0.287	0.002	6.125	6.432	0.45	224
2	6.125	0.035	6.125	12.285	0.86	112
3	1.531	2.25	3.063	6.844	0.479	112
4	3.063	9	3.063	15.125	1.059	56
5	0.766	4.5	1.531	6.797	0.476	56
6	1.531	18	1.531	21.063	1.474	56
7	1.531	18	1.531	21.063	1.474	56
8	0.383	9	0.766	10.148	3.02	28
9	0.766	36	0.766	37.531	2.627	14
10	0.766	36	0.766	37.531	2.627	14
11	0.191	9	0.191	9.383	0.657	14
12	0.191	9	0.191	9.383	0.657	14
13	0.191	9	0.191	9.383	0.657	14
Total	17.322	159.805	25.84	202.967	14.208	

with $7 \times n$ size for the ImageNet dataset, where n is denoted as in Table I. The input SRAM buffer has size enough to store 7×6 tiles with 1024 input channels for commonly used models. The proposed design will first compute all channels of an input tile with all weight data to get the final output tile. During this computation, all input data, partial sum, and output will access local buffer instead of external DRAM. Only the weight data will be accessed from DRAM repeatedly when the input tiles are changed. So, external DRAM access can be formulated as follows:

$$\text{Input Access} = \text{input channel} \times \text{image size} \quad (2)$$

$$\begin{aligned} \text{Weight Access} &= \text{number of input tiles} \\ &\quad \times \text{output channel} \times \text{input channel} \\ &\quad \times \text{weight size} \end{aligned} \quad (3)$$

$$\text{Output Access} = \text{output channel} \times \text{output size} \quad (4)$$

The total energy is $70\text{pJ/bit} \times \text{total access}$ for DDR3 DRAM [30]. As shown in Table. I, there are no redundant accesses in input and output. The weight accesses are correlated with the number of tiles. With the configurable tile size, we can tradeoff between tile size and input channel to decrease the number of tiles and weight accesses.

Fig. 15(a) shows power breakdown of the logic circuit in the core. SRAM accounts for 62% of the total power. The combinational logic consumes 13% for MAC and accumulator computations. The sequential logic consumes only 20% of

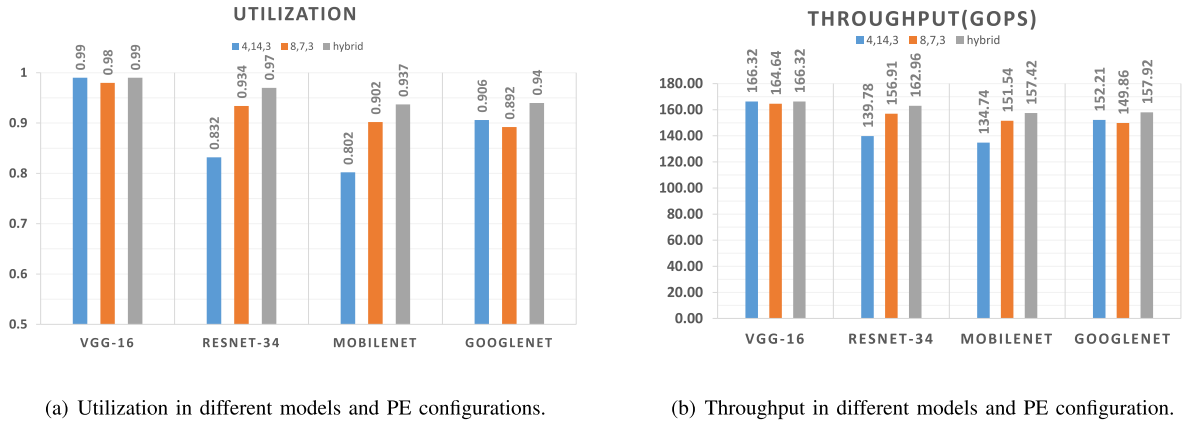


Fig. 21. Performance in different models and PE configurations.

TABLE II
IMPLEMENTATION RESULT AND COMPARISONS WITH OTHER DESIGNS

	Our work	[16]	[17]	[18]	[19]	[21]	[31]	[32]	[33]
Technology	40nm	65nm	65nm	65nm	28nm	65nm	28nm	65nm	65nm
Measurements	Post-layout	Chip	Post-layout	Post-layout	Chip	Post-layout	Post-layout	Chip	Post-layout
Supply Voltage (V)	0.99	1.0	1.0	1.2	0.65 - 1.1	1.0	-	1.2	1.2
Precision (bits)	16 fixed	16 fixed	8-20 fixed	16 fixed	4, 8, 16 Dyna.	16 fixed	16 fixed	10	16 fixed
PE number	168	168	192	512	-	168	168 / 374	128	64
Clock rate (MHz)	500	200	200	200	200	500	200	100	200
^a Peak Throughput (GOPS)	168 ^c 168	67.2 ^c 109.2	153.6 ^c 249.6	204.8 ^c 332.8	102 - 408 ^c 71.4 - 285.6	152 ^c 247	67.2 / 149.6 ^c 47 / 104.7	25.6 ^c 41.6	23.4 ^c 38.0
^b Area (KGE) (logic only)	266.9	1176	2695	-	-	1300	- / -	2103	282
Area (mm ²)	^e 1.5625	^e 12.25	-	^f 16.0	^e 1.87	^e 5.0	3.84 / 6.0	10.52	10.6
SRAM(KB)	191	181.5	192	280	144	96	80.5	312	139.6
^d Area eff. (GOPS/KGE)	0.629 ^c 0.629	0.057 ^c 0.092	0.057 ^c 0.092	-	-	0.117 ^c 0.19	- / - - / -	0.012 ^c 0.019	0.08 ^c 0.13
^d Area eff. (GOPS/mm ²)	107.52 ^c 107.52	5.485 ^c 8.914	-	12.8 20.8	54.5 - 218 ^c 38.1 - 152	30.4 ^c 49.4	17.5 / 24.9 ^c 12.2 / 17.5	2.43 ^c 3.95	2.20 ^c 3.58
^g Power (mW)	154.98	278	460.5	479	ⁱ 7.5 - ^j 300	354	375 / 535	20.5	93.4
Power eff. (TOPS/W)	1.084 ^h 1.084	0.241 ^h 0.399	0.333 ^h 0.552	0.406 ^h 0.969	^j 0.26 - ⁱ 10 ^h ^j 0.56 - ⁱ 3.017	0.429 ^h 0.711	0.18 / 0.27 -	1.24 ^h 2.96	0.25 ^h 0.596

^a 1 GMACS = 2 GOPS
^c Technology scaling ($\frac{process}{40nm}$)
^e Core only size.
^g Core only power.
ⁱ 30% - 60% sparsity, 3 - 4bits, and 76 GOPS at 0.65V.
^b The area is shown in terms of the size of kilo NAND2 gates (KGE).
^d We take the theoretical performance to evaluate area efficiency fairly here.
^f Chip size.
^h Normalized power efficiency = power efficiency $\times (\frac{process}{40nm}) \times (\frac{Voltage}{0.99V})^2$.
^j 16 bits precision and 76 GOPS at 1.1V.

3×3 convolution with stride 2 that have only 80% of utilization. The utilization of the last layer of the 3×3 convolutions with stride 2 is dropped to 58% due to small 7×7 feature maps. The final few layers have 49% of utilization for the (4, 14, 3) case due to small feature map size, 7, in large PE rows, 14, of a block. To improve hardware utilization, we reconfigure the PE array to be (4, 14, 3) for the first layer, and (8, 7, 3) for the last few layers with the layer adaptive PE configuration, and thus have the best one from (8, 7, 3) and (4, 14, 3) cases.

For MobileNet, the layers with 3×3 stride 1 have 100% of utilization. 1×1 layers have 89% of utilization since we only process $8 \times 3 = 24$ or $4 \times 3 = 12$ channels at a time and cannot have full hardware utilization for the certain number of channels. The layer with 3×3 stride 2 has lower hardware utilization with the same reason as above. In this case, we can also use layer adaptive PE reconfiguration as above for higher utilization. In summary, with the proposed data flow, we can attain high utilization while keep structure simple.

For GoogLeNet, utilization of the most layers with 3×3 and 1×1 achieves near 100% of utilization due to our reconfigurable PE array for different convolution kernels. Only the first one and last few layers have lower utilization due to the same reasons as in MobileNet. However, with our configurable PE blocks, we can attain higher utilization for these layers.

Fig. 21(a) shows the overall hardware utilization in different models and PE configurations. The utilization of 4 blocks is always lower than 8 blocks because hardware utilization is only half when the size of the feature map is down-sampled to 7 (half of 14 rows) for ResNet-34, GoogLeNet, and MobileNet. VGG-16 is the only exception since its minimum feature map size is 14. The layer adaptive PE configuration has the best utilization. With the proposed data flow, the utilization can achieve 99%, 97%, 93.7%, and 94% for VGG-16, ResNet-34, MobileNet and GoogLeNet, respectively.

TABLE III
LAYER BY LAYER COMPARISON FOR ALEXNET

	Our work	[16]	^a Ratio	[18]	^a Ratio
PE number	168	168	^b 1	512	^b 0.328
Clock rate	500MHz	200MHz	^b 2.5	200MHz	^b 2.5
Peak throughput (clock rate * PE)	168	67.2	^b 2.5	204.8	^b 0.82
Real throughput (GOPS)	146.33	46.314	^b 3.159	180.4	^b 0.811
CONV1 (ms)	1.515	5.225	^a 3.449	1.09	^a 0.719
CONV2 (ms)	3.348	10.475	^a 3.129	2.4	^a 0.717
CONV3 (ms)	1.92	5.9	^a 3.073	1.73	^a 0.901
CONV4 (ms)	1.44	4.6	^a 3.194	1.3	^a 0.903
CONV5 (ms)	0.96	2.625	^a 2.734	0.86	^a 0.896
Total Latency (ms)	9.184	28.825	^a 3.139	7.38	^a 0.804

^aRatio = compared work / our work for layer latency.
^bRatio = our work / compared work for other.

TABLE IV
LAYER BY LAYER COMPARISON FOR VGG-16

	Our work	[16]	Ratio
PE number	168	168	^b 1
Clock rate	500MHz	200MHz	^b 2.5
Peak throughput (clock rate * PE)	168	67.2	^b 2.5
CONV1 (ms)	5.161	25.4	^a 4.922
CONV2 (ms)	11.012	303.43	^a 27.555
CONV3 (ms)	5.506	156.77	^a 28.473
CONV4 (ms)	11.012	298.1	^a 27.071
CONV5 (ms)	5.506	80.37	^a 14.597
CONV6 (ms)	11.012	153.63	^a 13.951
CONV7 (ms)	11.012	152.57	^a 13.855
CONV8 (ms)	5.506	45.27	^a 8.222
CONV9 (ms)	11.012	84.93	^a 7.713
CONV10 (ms)	11.012	82.1	^a 7.456
CONV11 (ms)	2.75	18.1	^a 6.509
CONV12 (ms)	2.75	17.9	^a 6.509
CONV13 (ms)	2.75	17.9	^a 6.509
Total Latency (ms)	100.69	1436.47	^a 14.266

^aRatio = compared work / our work for layer latency.
^bRatio = our work / compared work for throughput.

Fig. 21(b) shows the overall throughput in different models and PE configurations. These throughput numbers are proportional to the hardware utilization since our models have optimized for different convolutional types in these models.

F. Design Comparison

Table. II shows the implementation result and comparison with other designs. The proposed design has much lower area cost than other designs due to the simpler PE structure and regular local summation. The peak throughput can reach up to 168 GOPS. The area efficiency is 0.629 GOPS/KGE and 107.52 GOPS/mm², which is much higher than other designs. The power consumption is 154.98mW and power efficiency can reach up to 1.084 TOPS/W because of the simple and regular data flow. The power efficiency is better than most of designs except for [19] and [32]. Reference [19] uses lower bitwidth hardware to compute network with 30% - 60% of sparsity at lower supply voltage to attain lower area cost or power consumption. Reference [32] uses an optimized numerical representation for lower bitwidth hardware to achieve lower power consumption.

Table III shows a layer-by-layer performance comparison for AlexNet. For a fair comparison, we first list speedup ratio of the peak throughput between different designs and the proposed work. This can normalize the effects of different clock rate and PE numbers in each design, which implies a full hardware utilization case. Then, for each layer, if the compared design has lower hardware utilization than the proposed one, the speedup ratio will become larger. Otherwise, the speedup ratio will be lower. By dividing speedup of the layers with speedup of the peak throughput, we will know the difference in hardware utilization between designs. Thus, compared to [16], the hardware utilization of our design is 20.4% ($3.139 - 2.5/3.139$) higher. When compared to [18], the hardware utilization of our design 1.95% ($0.804 - 0.82/0.82$) lower since our design has lower hardware utilization for CNN with low number of channels and non- 3×3 convolutions at the first and second layers of AlexNet, respectively. However, this difference is quite small. With our 3×3 optimized based structure, we can have the simpler structure and lower cost compared to [18]. Beyond the first two layers, the utilization of our design is 12% higher at other layers, which proves the efficiency of the proposed design.

Table IV shows another comparison for VGG-16. The utilization of proposed design has 470% higher hardware utilization than that in [16]. The reason is that these layers are our optimized 3×3 layers and have 100% of hardware utilization. The only exception is the first layer due to only three input channels. The large hardware utilization degradation at CONV1 to CONV7 in [16] is due to bad handling of the large feature map in their PE structure. On the contrary, the broadcasted vectorwise input approach in our design can keep PE hardware utilization high.

VI. CONCLUSION

This paper proposes a hardware efficient CNN accelerator that can execute different convolution filters efficiently, such as 3×3 , 4×4 , 5×5 , 3×3 stride 2, depthwise, and 1×1 convolutions. This flexible scheduling is enabled by layer adaptive PE configurations, interleaved input selection scheme, elementwise input, and reconfigurable accumulators. To reduce cost but still meet above flexibility requirements, we adopt a systolic array architecture with vectorwise input and weight

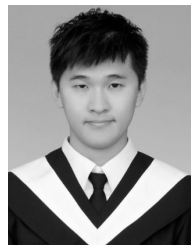
data to keep structure regular. The proposed design can achieve at least 93.7% of hardware utilization for commonly used image recognition networks, with 266.9K NAND gate counts, 191KB SRAM buffer, and $1.25\text{mm} \times 1.25\text{mm}$ core area when implemented on TSMC 40nm CMOS process, which has higher efficiency than existing designs.

ACKNOWLEDGMENT

The authors would like to thank TSRI for its support with EDA design tools.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. NIPS*, 2012, pp. 1097–1105.
- [2] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. CVPR*, Jun. 2015, pp. 1–9.
- [3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.
- [5] S. Ji, W. Xu, M. Yang, and K. Yu, "3D convolutional neural networks for human action recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 1, pp. 221–231, Jan. 2013.
- [6] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun, "OverFeat: Integrated recognition, localization and detection using convolutional networks," in *Proc. ICLR*, 2014, pp. 1–16.
- [7] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. CVPR*, Jun. 2014, pp. 580–587.
- [8] T. He, W. Huang, Y. Qiao, and J. Yao, "Text-attentional convolutional neural network for scene text detection," *IEEE Trans. Image Process.*, vol. 25, no. 6, pp. 2529–2541, Jun. 2016.
- [9] H. Li, Z. Lin, X. Shen, J. Brandt, and G. Hua, "A convolutional neural network cascade for face detection," in *Proc. CVPR*, Jun. 2015, pp. 5325–5334.
- [10] D. Tomè, F. Monti, L. Baroffio, L. Bondi, M. Tagliasacchi, and S. Tubaro, "Deep convolutional neural networks for pedestrian detection," *Signal Process., Image Commun.*, vol. 47, pp. 482–489, Sep. 2016.
- [11] T. Chen *et al.*, "DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," in *Proc. ASPLOS*, Mar. 2014, pp. 269–284.
- [12] Y. Chen *et al.*, "Dadiannao: A machine learning supercomputer," in *Proc. MICRO*, Dec. 2014, pp. 609–622.
- [13] Z. Du *et al.*, "ShiDianNao: Shifting vision processing closer to the sensor," in *Proc. ISCA*, Jun. 2015, pp. 92–104.
- [14] D. Liu *et al.*, "PuDianNao: A polyvalent machine learning accelerator," in *Proc. ASPLOS*, Mar. 2015, pp. 369–381.
- [15] T. Chen *et al.*, "A high-throughput neural network accelerator," *IEEE Micro*, vol. 35, no. 3, pp. 24–32, May/Jun. 2015.
- [16] Y. H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [17] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE J. Emerging Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, Jun. 2019.
- [18] F. Tu, S. Yin, P. Ouyang, S. Tang, L. Liu, and S. Wei, "Deep convolutional neural network architecture with reconfigurable computation patterns," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 8, pp. 2220–2233, Aug. 2017.
- [19] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst, "14.5 Envision: A 0.26-to-10TOPS/W subword-parallel dynamic-voltage-accuracy-frequency-scalable Convolutional Neural Network processor in 28nm FDSOI," in *Proc. IEEE ISSCC*, Feb. 2017, pp. 246–247.
- [20] Y.-J. Lin and T. S. Chang, "Data and hardware efficient design for convolutional neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 5, pp. 1642–1651, May 2018.
- [21] L. Du *et al.*, "A reconfigurable streaming deep convolutional neural network accelerator for Internet of Things," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 1, pp. 198–208, Jan. 2018.
- [22] Y. Huan, J. Xu, L. Zheng, H. Tenhunen, and Z. Zou, "A 3D tiled low power accelerator for convolutional neural network," in *Proc. IEEE ISCAS*, May 2018, pp. 1–5.
- [23] D. Im, D. Han, S. Choi, S. Kang, and H.-J. Yoo, "DT-CNN: Dilated and transposed convolution neural network accelerator for real-time image segmentation on mobile devices," in *Proc. IEEE ISCAS*, May 2019, pp. 1–5.
- [24] S. Liu *et al.*, "Cambricon: An instruction set architecture for neural networks," in *Proc. ISCA*, Jun. 2016, pp. 393–405.
- [25] S. Zhang *et al.*, "Cambricon-X: An accelerator for sparse neural networks," in *Proc. MICRO*, Oct. 2016, p. 20.
- [26] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, Sep. 2014, pp. 1–14.
- [27] A. Howard *et al.*, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [28] C. Szegedy, S. Ioffe, and V. Vanhoucke, "Inception-v4, inception-resnet and the impact of residual connections on learning," 2016, *arXiv:1602.07261*. [Online]. Available: <https://arxiv.org/abs/1602.07261>
- [29] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural network with pruning, trained quantization and Huffman coding," in *Proc. ICLR*, Feb. 2016, pp. 1–14.
- [30] K. T. Malladi, F. A. Nothaft, K. Periyathambi, B. C. Lee, C. Kozyrakis, and M. Horowitz, "Towards energy-proportional datacenter memory with mobile dram," in *Proc. ISCA*, Jun. 2012, pp. 37–48.
- [31] H. Kwon, A. Samajdar, and T. Krishna, "MAERI: Enabling flexible dataflow mapping over DNN accelerators via reconfigurable interconnects," in *Proc. ASPLOS*, Mar. 2018, pp. 461–475.
- [32] Y. Choi, D. Bae, J. Sim, S. Choi, M. Kim, and L.-S. Kim, "Energy-efficient design of processing element for convolutional neural network," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 64, no. 11, pp. 1332–1336, Nov. 2017.
- [33] J. Jo, S. Kim, and I.-C. Park, "Energy-efficient convolution architecture based on rescheduled dataflow," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 65, no. 12, pp. 4196–4207, Dec. 2018.
- [34] A. Ardakani, C. Condo, M. Ahmadi, and W. J. Gross, "An architecture to accelerate convolution in deep neural networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 65, no. 4, pp. 1349–1362, Apr. 2018.
- [35] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 217–230, Jan. 2019.
- [36] J. Wang, J. Lin, and Z. Wang, "Efficient hardware architectures for deep convolutional neural network," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 65, no. 6, pp. 1941–1953, Jun. 2017.
- [37] A. Canziani, E. Culurciello, and A. Paszke, "An analysis of deep neural network models for practical applications," 2017, *arXiv:1605.07678*. [Online]. Available: <https://arxiv.org/abs/1605.07678>



Kuo-Wei Chang is currently pursuing the Ph.D. degree with National Chiao Tung University (NCTU), Hsinchu, Taiwan, with a focus on VLSI design and hardware implementation for deep learning accelerators.



Tian-Sheuan Chang (S'93–M'06–SM'07) received the B.S., M.S., and Ph.D. degrees in electronic engineering from National Chiao Tung University (NCTU), Hsinchu, Taiwan, in 1993, 1995, and 1999, respectively.

From 2000 to 2004, he was a Deputy Manager with Global Unichip Corporation, Hsinchu. In 2004, he joined the Department of Electronics Engineering, NCTU, where he is currently a Professor. In 2009, he was a Visiting Scholar with imec, Belgium. His current research interests include system-on-a-chip design, VLSI signal processing, and computer architecture.

Dr. Chang was named the Excellent Young Electrical Engineer by the Chinese Institute of Electrical Engineering in 2007 and the Outstanding Young Scholar by the Taiwan IC Design Society in 2010. He has been actively involved in many international conferences as an organizing committee or technical program committee member.